
CSC 412 – Operating Systems

Lab Session 06, Fall 2025

Tuesday, October 14th and Thursday, October 16th, 2025

What This Lab Is About

This lab could be titled: “C++ vectors/lists meet structs,” because this is what we are going to do. So, yes, now after a bit of time doing C programming to insist on memory allocation, pointer algebra, and typecasting, we are back with C++. From now on, all labs and assignments will use C++. This means not only that `const` and `bool` are fine again, but that you encouraged to use convenient data types (e.g. vectors and lists, as in this lab). Don’t take this as an encouragement to go wild with OPP, though, as this is *definitely not* the purpose of this course.

While I was quite conservative regarding C programming, advocating that you stick to the pretty old “baseline C” standard of C99, I encourage you to use modern features of C++. As of today, the latest stable version of C++ supported on all platforms is C++23, so you should build your programs with the `-std=c++23` option (and of course still the `-Wall` option).

Handout

For this lab, I give you a small initial program `lab06.cpp` that defines a `struct` data type, as well as a data file you are supposed to read from.

1 Task 1: Build a vector of structs

The handout for this lab reminds you of how to read data from a file. Instead of simply printing out the data, build a vector of structs and then iterate through your vector to print out the fields of the structs.

2 Task 2: Same thing with a list of structs

Instead of using a vector, use a list to store the struct that you read from the data file. Make sure to use a `const` iterator when you print out the content of your list.

3 Task 3: Partial list/vector

Let’s say that we want to maintain a separate sublist (or sub-vector; your choice) of the one we built from our data file. For example, it is pretty common to store a list of “selected” structs that must undergo some special processing.

In this case, let's say that we want to build a list of all structs that have an even `m` field (granted, this is a completely artificial task. This is just to provide a simple selection criterion).

Iterating through your “selected” list, multiply the `x` field of all objects by a factor 10, then print again the list of all structs.

Note: Be careful that a list/vector only stores a *copy* of the object. If you modify a copy, the original will not be properly updated.